

Compile-time Object-Relational Mapping

библиотека на C++

Дипломна работа

Алекс Цветанов, фак. № 121222225

Научен ръководител: доц. д-р инж. Галя Павлова

Технически университет -- София
Факултет „Компютърни системи и технологии“
Компютърно и софтуерно инженерство

2026

Съдържание

- 1 Мотивация
- 2 Архитектура
- 3 Реализация
- 4 Backend сценарии
- 5 Верификация
- 6 Приноси и заключение

Проблемът с runtime заявките

Традиционен подход:

- Заявките се описват като **НИЗОВЕ**
- Валидиране едва при изпълнение
- Грешни имена на полета → runtime crash
- Несъвместими типове → late discovery
- Смяна на backend → **пълно пренаписване**

Изследователски въпрос:

Може ли компилаторът да поеме ролята на първи валидатор на заявките?

Runtime

Низ заявки

Грешка при
изпълнение

Late discovery

Compile-time

`constexpr` IR

Грешка при
компиляция

Early detection

Цел

Компилаторът поема ролята на първи валидатор на заявките.

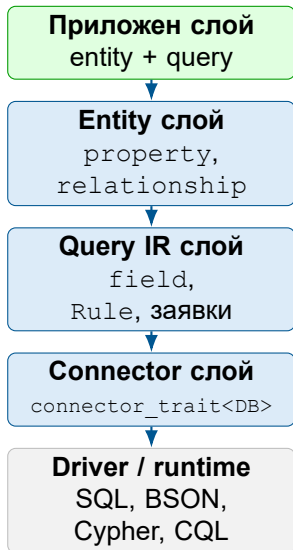
Съществуващи решения и позиция на разработката

Решение	Подход	Ограничение
SOCI	runtime bind	низ-базирани заявки
ODB	codegen	external compiler
sqlpp11	EDSL	само SQL
Hiberlite	ORM	само SQLite

Приносът на разработката

- + `constexpr` query IR без codegen
- + Multi-backend: SQL и NoSQL
- + Compile-time capability проверки
- + Формален connector contract

Слоеста архитектура на библиотеката



Ключови архитектурни принципи:

- Никакъв виртуален интерфейс trait-базиран, не наследствен contract
- Query IR е `constexpr` значима валидация при компилация
- Capability tags ограничават неподдържаните операции
- Смяна на backend = смяна само на типа на конектора

Централен инвариант

Приложният код не комуникира директно с драйвера — работи само с C++ типове.

Entity модел и Query IR

Entity описание:

```
struct User {
    orm::property<int,          "id">    id;
    orm::property<std::string, "name"> name;
    orm::property<double,      "score"> score;
};

template<> struct orm::table_name_trait<User> {
    static constexpr std::string_view
        value = "users";
};
```

`property<T, "col">` носи тип, compile-time name и member pointer — без macro, без codegen.

Query IR конструкции:

```
// field<Ptr> -- wrapper над member ptr
// Rule       -- типизиран предикат
// Placeholder -- анонимен параметър

auto pred =
    orm::field<&User::id>
        == orm::Placeholder<int>{};

// constexpr обект
static constexpr auto q =
    orm::select(
        orm::field<&User::id>,
        orm::field<&User::name>
    ).where(pred);
```

Изграждане на сложна заявка

```
using namespace orm::literals;

static constexpr auto q =
    orm::select(
        orm::field<&User::id>,
        orm::field<&User::name>,
        orm::field<&User::score>)
    .where(
        orm::field<&User::score> > 3.14)
    .where(
        orm::field<&User::id>
            == orm::Placeholder<int>{})
    .order_by<orm::order::direction::desc>(
        orm::field<&User::score>)
    .limit(10_per_page & 1_page);

static_assert(
    decltype(q.where_clauses())::size == 2);
```

Характеристики:

- Fluent API — всяка стъпка връща нов типизиран обект
- `static constexpr` — IR живее в code сегмента; нулев runtime overhead
- `static_assert` проверява структурата при компилация
- `group_by`, `join<>` — аналогичен синтаксис

Pagination UDL

`10_per_page & 2_page`

двата оператора са взаимозаменяеми

Connector trait и Capability механизъм

Минимален connector contract:

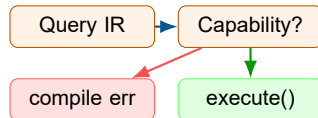
```
template<>
struct orm::connector_trait<MyDB> {
    // Задължителни типове
    template<typename T> struct wire_type;
    using cursor_type = ...;

    // Capability тагове
    using supports_joins      = true_type;
    using supports_transactions = true_type;

    // Materialization entry point
    static auto execute(
        MyDB& db, auto query,
        auto&&... params);
};
```

Compile-time capability:

- `is_connector<DB>` concept
проверява всеки тип
- JOIN без `supports_joins` →
compile error
- Транзакция без
`supports_transactions` →
compile error



Multi-backend материализация

Един Query IR → различни backend-и:

Backend	Материализация
SQLite	WHERE id = ?
PostgreSQL	WHERE id = \$1
MySQL	WHERE id = ?
MongoDB	BSON filter
Redis	GET/SET/DEL
Neo4j	Cypher MATCH...WHERE
Cassandra	CQL SELECT...FROM

Абстракция на връзките:

embed	MongoDB вложен документ
reference	SQL foreign key / JOIN
	Redis: key prefix
	Neo4j: edge

Принцип

C++ моделът е единственият *логически* носител на схемата. Backend-ът определя физическата материализация.

```
static constexpr auto q =
    orm::select(orm::field<&User::name>
        .where(orm::field<&User::id>
            == orm::Placeholder<int>{}));

auto stmt = db.prepare(q);
auto r1 = stmt.execute(42);
auto r2 = stmt.execute(99);

// Индексирани placeholders-
using namespace orm;
static constexpr auto q2 =
    orm::select(orm::field<&User::name>
        .where(orm::field<&User::id>
            == ph<int, _1>)
        .where(orm::field<&User::score>
            > ph<double, _1>);
```

Характеристики:

- `prepare()` → `prepared_query<Q, DB>`
- Многократна употреба без повторно рендиране
- `ph<T, _N>` — индексирани placeholder; `?N` (SQLite) или `$N` (PostgreSQL)
- Един runtime параметър може да запълни повече от един слот

Верифицирано в

```
test_mockdb.cpp:
PrepareReturnsExecutableObject
IndexedPlaceholderReused...
```

Верификационна стратегия

Тестова инфраструктура

214 теста — 100% преминаване
Windows/MSVC и Linux/GCC 13 в Docker

Покритие:

- Query IR изграждане и структура
- Capability-базирано ограничаване
- Подготвени заявки и placeholder-и
- Schema миграции (`migrate<DB>`)
- Нишкова безопасност
- Live integration тестове

Нива на верификация:

- 1 **Compile-time:** `static_assert`, concept constraints
- 2 **Unit:** изолирани компоненти с MockDB
- 3 **Integration:** SQL rendering, live бази данни
- 4 **Docker CI:** пълен suite на Linux

Ключов принцип

Не е достатъчно само „да има тестове“ или само compile-time checks.
Необходима е комбинация.

- ❶ **Практически приложим Compile-time ORM модел**
без external codegen, без виртуален интерфейс, в стандартен C++23
- ❷ **C++ entity описанието като единствен логически носител на схемата**
един модел → таблица / документ / key-value / граф / wide-column ред
- ❸ **Формален connector contract за разширяемост**
нов backend без промяна в query API; capability tags правят ограниченията видими при компилация
- ❹ **Многостепенна верификационна стратегия**
concept constraints + unit + integration + live

Заклучение

Постигнато:

- Query IR като `constexpr` обект — без низове, без runtime reflection
- 7 backend-a: SQLite, PG, MySQL, MongoDB, Redis, Neo4j, Cassandra
- 214 теста, Docker CI, два компилатора
- Connector contract формализиран и демонстриран с реални конектори

Бъдещо развитие:

- C++26 static reflection → автом. entity описание
- Distributed connection pools
- Автом. migration за всички backend-и
- Performance benchmarking

Основен извод

Между runtime ORM и driver-only съществува жизнеспособна арх. позиция:

Compile-time ORM слой в C++.

Благодаря за вниманието!

Въпроси?